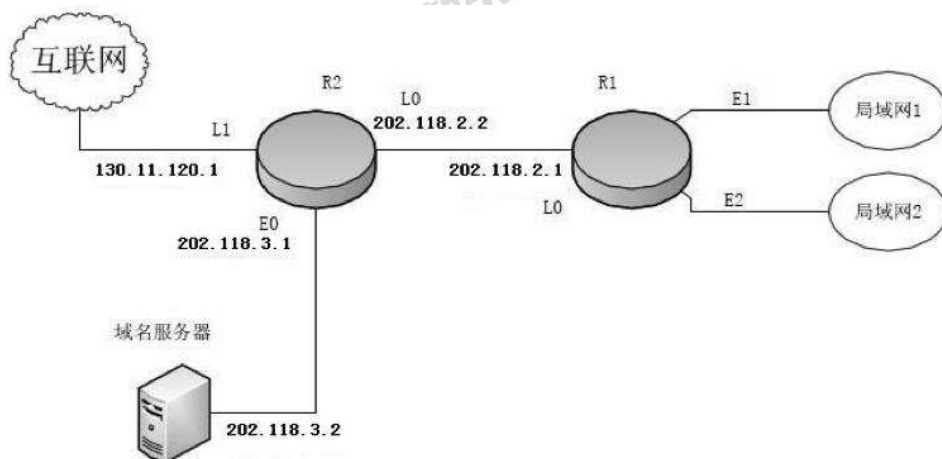


为 0 表示页面不在内存，产生缺页中断，缺页中断处理后，返回到产生缺页中断的指令处重新执行。设有虚地址访问序列 2362H、1565H、25A5H，请问：

(1) 依次访问上述三个虚地址，各需多少时间？给出计算过程。

(2) 基于上述访问序列，虚地址 1565H 的物理地址是多少？请说明理由。

47. (9 分) 某网络拓扑如下图所示，路由器 R1 通过接口 E1、E2 分别连接局域网 1、局域网 2，通过接口 L0 连接路由器 R2，并通过路由器 R2 连接域名服务器与互联网。R1 的 L0 接口的 IP 地址是 202.118.2.1；R2 的 L0 接口的 IP 地址是 202.118.2.2，L1 接口的 IP 地址是 130.11.120.1，E0 接口的 IP 地址是 202.118.3.1；域名服务器的 IP 地址是 202.118.3.2。



R1 和 R2 的路由表结构为：

目的网络 IP 地址	子网掩码	下一跳 IP 地址	接口
------------	------	-----------	----

(1) 将 IP 地址空间 202.118.1.0/24 划分为 2 个子网，分别分配给局域网 1、局域网 2，每个局域网需分配的 IP 地址数不少于 120 个。请给出子网划分结果，说明理由或给出必要的计算过程。

(2) 请给出 R1 的路由表，使其明确包括到局域网 1 的路由、局域网 2 的路由、域名服务器的主机路由和互联网的路由。

(3) 请采用路由聚合技术，给出 R2 到局域网 1 和局域网 2 的路由。

2009 年计算机学科专业基础参考答案

一、单项选择题

1. 【解析】B。缓冲区的概念出现在操作系统的设备管理中，其特点是先进先出。缓冲区的作用是解决主机与打印机之间速度不匹配的问题，而不应改变打印数据的顺序。若用栈，先进入缓冲区的数据则要排队到最后才能打印，显然不符题意，故选 B。

2. 【解析】C。即出栈顺序是 b, d, c, f, e, a, g，结合入栈顺序 a, b, c, d, e, f, g，栈内元素最多为(a, c, d), (a, e, f)。元素的出栈顺序是 b, d, c, f, e, a, g，可推出进栈出栈顺序为 Push(S, a), Push(S, b), Pop(S, b), Push(S, c), Push(S, d), Pop(S, d), Pop(S, c), Push(S, e), Push(S, f), Pop(S, f), Pop(S, e), Pop(S, a), Push(S, g), Pop(S, g)。假设初始所需容量为 0，每做一次 Push 进行一次“+1”操作，每做一次 Pop 进行一次“-1”操作，

记录容量的最大值为 3, 所以选 C。

3.【解析】D。分析遍历后的结点序列, 可以看出根结点是在中间访问, 而右子树结点在左子树之前, 即遍历的方式是 RNL。本题考查的遍历方法并不是二叉树的 3 种基本遍历方法, 对于考生而言, 重要的是要掌握遍历的思想。

4.【解析】B。根据平衡二叉树的定义有, 任意结点的左、右子树高度差的绝对值不超过 1。而其余 3 个选项均可以找到不符合该条件的结点。在做题过程中, 如果答案不太明显, 可以把每个非叶结点的平衡因子都写出来再进行判断。

5.【解析】C。完全二叉树比满二叉树只是在最下面一层的右边缺少了部分叶结点, 而最后一层之上是个满二叉树, 并且只有最后两层有叶结点。第 6 层有叶结点则完全二叉树的高度可能为 6 或 7, 显然树高为 7 时结点更多。若第 6 层上有 8 个叶结点, 则前六层为满二叉树, 而第 7 层缺失了 $8 \times 2 = 16$ 个叶结点, 故完全二叉树的结点个数最多为 $(2^7 - 1) - 16 = 111$ 个结点。

6.【解析】B。森林与二叉树的转换规则为“左孩子右兄弟”。在最后生成的二叉树中, 父子关系在对应森林关系中可能是兄弟关系或原本就是父子关系。情形 I: 若结点 v 是结点 u 的第二个孩子结点, 在转换时, 结点 v 就变成结点 u 第一个孩子的右孩子, 符合要求。情形 II: 结点 u 和 v 是兄弟结点的关系, 但二者之中还有一个兄弟结点 k , 则转换后, 结点 v 就变为结点 k 的右孩子, 而结点 k 则是结点 u 的右孩子, 符合要求。情形 III: 若结点 u 的父结点与 v 的父结点是兄弟关系, 则转换后, 结点 u 和 v 分别在两者最左父结点的两棵子树中, 不可能出现在同一条路径中。

7.【解析】A。每条边都连接了两个结点, 在计算顶点的度之和时每条边都被计算了两次 (出度和入度), 故所有顶点的度之和为边数的两倍, I 正确。 n 个顶点、 $n-1$ 条边可以构成无向连通图, 比如树, II 错误。顶点数为 $N(N \geq 1)$ 的无向完全图中不存在度为 1 的顶点, III 错误。

8.【解析】D。选项 A、B 和 C 都是 B-树的特点, 而选项 D 则是 B+树的特点。注意区别 B-树和 B+树各自的特点。

9.【解析】A。根据小根堆调整规则调整即可。插入关键字 3 时, 先将其放在小根堆的末端, 再将该关键字向上进行调整, 调整后的小根堆序列为 3, 5, 12, 8, 28, 20, 15, 22, 19。

10.【解析】B。冒泡和选择排序每趟之后都会有一个元素被放置在最终位置上。 $\{11, 12\}$ 和 $\{4, 5\}$ 并不是最终位置, 因此排除冒泡和选择。二路归并算法之后应该是每 4 个元素有序。但是 $\{11, 12, 13, 7\}$ 并不有序, 因此排除二路归并算法。解答本题需要对各种排序算法的特点极为清楚。对于冒泡排序和选择排序, 每一趟都能确定一个元素的最终位置, 而题目中, 前 2 个元素和后 2 个元素均不是最小或最大的 2 个元素并按序排列。选项 D 中的 2 路归并排序, 第一趟排序结束都可以得到若干个有序子序列, 而此时的序列中并没有两两元素有序排列。插入排序在每趟排序后能确定前面的若干元素是有序的, 而此时第二趟排序后, 序列的前三个元素是有序的, 符合其特征。

11.【解析】C。虽然指令和数据都是以二进制存放在存储器中的, 但 CPU 可以根据指令周期的不同阶段来区分指令还是数据, 通常在取指令阶段取出的是指令, 执行阶段取出的是数据。此题容易误选 A, 需要明白的是, CPU 只有在确定取出的是指令之后, 才会将操作码送去译码, 因此不能根据译码的结果来区分指令和数据。

12.【解析】D。int 为 32 位，short 为 16 位；C 语言的数据在内存中为补码形势，所以 x、y 的机器数写为 0000 007F、FFF7H。之行 $z=x+y$ 是，由于 x 为 int 型，y 为 short 型，需要将 y 强制转换成 int 型。由于 y 的符号位是 1，所以在 y 之前添加 16 个 1，再执行加法：0000 007FH + FFFF FFF7H = 0000 0076H，最高位的进位 1 丢弃。

13.【解析】D。浮点数加法步骤：对阶；尾数相加；规格化；判溢出。B、C，没有规格化和判溢出。X 的浮点数格式为 00, 111; 00, 11101（分号前为阶码，分号后为尾数）。Y 的浮点数格式为 00, 101; 00, 10100，然后根据浮点数的加法步骤进行运算。

第一步：对阶。X、Y 阶码相减， $00\ 111-00\ 101=00\ 111+11\ 0111=00\ 010$ 。可知 X 的阶码比 Y 的阶码大 2（可直接目测出来）。小阶向大阶看齐，Y 的阶码加 2，尾数右移两位，Y 变为 00 111; 00 00101。

第二步：尾数相加。 $00\ 11101+00\ 00101=01\ 00010$ 。结果符号位为 01，需要右规。

第三步：规格化。尾数右移一位，阶码加一，得 X+Y 为 01 000; 00 10001。

第四步：判断溢出。阶码符号位为 01，说明发生溢出。

14.【解析】C。共分为 8 组，组号为 0、1、2、...、7。主存的某一字块按模 8 映射到 Cache，即主存的第 0，8，16...字块可以映射到 Cache 第 0 组的任一字块中。每个主存块为 32 字节，所以 129 号单元位于第 4 块主存块。

15.【解析】D。首先确定 ROM 的个数，ROM 区为 4KB，选用 2K*8K 的 ROM，需要 $(4K*8)/(2K*8)=2$ 片，采用字扩展方式；60KB 的 RAM 区，选用 4K*4 位的 RAM 芯片，需要 $(60K*8)/(4K*4)=30$ 片，采用字和位同时扩展方式。

16.【解析】C。相对寻址 $EA=(PC)+A$ ，首先要求得是取指令后 PC 得值。转移指令由两个字节组成，每取一个字节 $PC+1$ ，所以 $PC=2002H$ ， $EA=2008H$ 。注意转移指令得大小以及 pc 的值先自增。

17.【解析】A。相对于 CISC，RISC 的特点是指令条数少；指令长度固定，指令格式和寻址种类少；只有取数/存数指令访问存储器，其余指令的操作均在寄存器之间进行；CPU 中通用寄存器多；大部分指令在一个或者小于一个指令周期内完成；以硬布线逻辑为主，不用或者少用微程序控制。

18.【解析】A。流水线的始终周期应当以最长的执行时间为准，否则时长较长的功能段将不能完成。

19.【解析】D。硬布线控制器采用专门的逻辑电路实现，其速度主要取决于逻辑电路的延迟，指令执行速度快，指令功能的修改和扩展难；微程序控制器采用存储程序的原理，每条机器指令对应一个微程序，指令执行速度慢，指令功能的修改和扩展快。

20.【解析】B。总线带宽是指单位时间内总线上传输数据的位数，通常用每秒钟传送信息的字节数来衡量，单位 Bis。由题意可知，在 1 个总线周期（等于 2 个时钟周期）内传输了 4 字节信息，时钟周期 $=1/10MHz=0.1\mu s$ ，故总线带宽为 $4B/(2*0.1\mu s)=4B/(0.2*10^{-6}s)=20MB/s$ 。

21.【解析】D。命中率 = Cache 命中的次数 / 总访问的次数。 $(1000-50)/1000*100\%=95\%$ 。命中率 = Cache 命中次数 / 总访问次数。需要注意的是看清题，题中说明的是缺失 50 次，而不是命中 50 次，仔细审题是做对题的第一步。

22.【解析】A。外部中断是指来自处理器和内存以外的部件引起的中断。内中断主要是指在处理器和内存内部产生的中断。A 中键盘输入属于外部事件，每次键盘输入 CPU 都需要执行中断以读入输入数据，所以能引起外部中断。B 中除数为 0 属于异常，内中断，发生在 CPU 内部。C 中浮点数运算下溢将按机器 0 处理，不会产生中断。D 中访存缺页属于 CPU 执行指令时发生的中断，不属于外部中断。所以能产生外部中断的只能是输入设备键盘。

23.【解析】D。在单处理机系统中，为提高系统效率，让内存中保持多个处于不同状态的进程（这称为进程并发执行），但在同一时刻占用处理机的却只有一个进程，因此，进程之间不是并行的；现代操作系统的形成正是通道产生的结果，通道是可以独立于 CPU 控制输入输出的设备，在处理机进行运算时，通道可同时进行 I/O 操作，因此，这两者也是并行的；处理机运算时，外设可同时进行输入/输出的操作，因此，也产生了并行；显然，设备和设备之间也是可以并行的。

24.【解析】D。本题考查操作系统调度算法的基础知识。高响应比优先调度算法的基本思想是把 CPU 分配给就绪队列中响应比最高的进程。高响应比优先调度算法既考虑作业的执行时间也考虑作业的等待时间，综合了先来先服务和最短作业优先两种算法的特点。该算法中的响应比是指作业等待时间与运行比值，响应比公式定义如下：响应比 = (等待时间 + 要求服务时间) / 要求服务时间，即 $RR = (w + s) / s = 1 + w / s$ ，因此响应比一定是大于 1 的。

25.【解析】C。每个进程 3 台，不会产生死锁；对于三个进程，可以有两个进程分别获得 3 台，使其执行完释放后让第三个进程获得 3 台，所以也不会产生死锁；对于四个进程，假若每个进程各获得 2 台而同时需要另外一台，产生了死锁，所以产生死锁的最小值是 4。

26.【解析】A。界地址寄存器来保护内存管理方式的主要措施。每个进程都拥有自己独立的进程空间，若一个进程在运行时所产生的地址在其地址空间之外，则发生地址越界，因此需要进行界地址保护，即当程序要访问某个内存单元时，由硬件检查是否允许，如果允许则执行，否则产生地址越界中断。

27.【解析】C。分段存储管理的逻辑地址分为段号 + 位移量。段内位移最大值就是最大段长。地址长度为 32 为，段号占 8 为，位移为 $32 - 8 = 24$ 位，最大段长为 $2^{24}B$ 。

28.【解析】B。链式不利于随机访问，连续不利于文件扩展。文件的物理结构包括连续、链式、索引三种，其中链式结构不能实现随机访问，连续结构的文件不易于扩展。因此随机访问且易于扩展是索引结构的特性。

29.【解析】A。根据 SCAN 调度算法可得。SCAN 算法类似电梯的工作原理。首先，当磁头从 105 道向序号增加的方向移动时，便会按照从小到大的顺序服务所有大于 105 的磁道号 (110, 170, 180, 195)；往回移动时又会按照从大到小的顺序进行服务 (68, 45, 35, 12)。

30.【解析】A。FCB 通常包含：基本信息、存取控制信息、使用信息。为了实现“按名存取”，在文件系统中为每个文件设置用于描述和控制文件的数据结构，称之为文件控制块 (FCB)。在文件控制块中，通常包含以下三类信息，即基本信息、存取控制信息及使用信息。

31.【解析】B。建立符号链接时，引用计数值直接复制；建立硬链接时，引用计数值加 1。建立 F2 时，F1 和 F2 的引用计数值都是 1。再建立 F3 时，F1 和 F3 的引用计数值都为 2。删除 F1 时，F3 的引用计数值为 $2 - 1 = 1$ ，F2 的引用计数值不变。

32.【解析】A。设备管理具有设备独立性的特点，将所有设备都看作逻辑资源，操作系统提供使用接口。应用程序使用逻辑设备名来请求使用某类设备，而系统中必须设置一张逻辑设备表，其中每个表目中包含了逻辑设备名、物理设备名和设备驱动程序入口地址。

33.【解析】B。传输层提供应用进程间的逻辑通信（通过端口号），即端到端的通信。而数据链路层负责相邻结点之间的通信，这个结点包括了交换机和路由器等数据通信设备，这些设备不能称为端系统。网络层负责主机到主机的逻辑通信。因此选 B。

34.【解析】B。采用 4 个相位，每个相位有 4 种幅度的 QAM 调制方法，每个信号可以有 16 种变化，传输 4bit 的数据。根据奈奎斯特定理，信息的最大传输速率为 $2 \times 3k \times 4 = 24\text{kbps}$ 。

35.【解析】C。在后退 N 帧协议中，当接收方检测到某个帧出错后，则简单地丢弃该帧及其后所有的后续帧，发送方超时后需重传该数据帧及其后续的所有帧。这里应注意，连续 ARQ 协议中，接收方一般采用累积确认的方式，即接收方对按序到达的最后一个分组发送确认，因此本题中收到 3 的确认帧就表示编号为 0, 1, 2, 3 的帧已接收，而此时发送方未收到 1 号帧的确认只能代表确认帧在返回的过程中丢失了，而不代表 1 号帧未到达接收方。因此需要重传的帧为编号是 4, 5, 6, 7 的帧。

36.【解析】A。数据链路层是用物理地址进行转发，而转发到目的地需要使用目的地址。因此 PDU 地址是目的物理地址。

37.【解析】D。若最短帧长减少，而数据传输速率不变，则需要使冲突域的最大距离变短来实现碰撞窗口的减少。碰撞窗口是指网络中收发结点间的往返时延，因此假设需要减少的最小距离为 S，则可以得到如下公式（注意单位的转换）：减少的往返时延 = 减少的发送时延，即 $2 \times [S / (2 \times 10^8)] = 800 / (1 \times 10^9)$ 的。即，由于帧长减少而缩短的发送时延，应等于由于距离减少而缩短的传播时延的 2 倍。可得 $S = 80$ ，即最远的两个站点之间的距离最少需要减少 80m。

38.【解析】D。第一个段的序列号为 200，两个 TCP 段长度为 800 字节，所以最后的确认序列号为 1000。

39.【解析】C。发生超时后，慢开始门限变为 16KB，接下来 3 个 RTT 内，执行慢开始算法，拥塞窗口分别为 2KB, 4KB, 8KB，由于慢开始门限为 8KB，因此之后转而执行拥塞避免算法。因此第 4 个 RTT 结束后，拥塞窗口为 9KB。

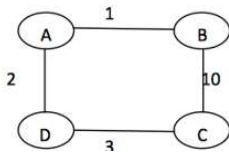
40.【解析】A。FTP 需要保证可靠性，并且使用了一个分离的控制连接传送命令。对于 FTP 文件传输，为了保证可靠性，选择 TCP 协议，排除 C、D。FTP 的控制信息是带外传送的，也即 FTP 使用了一个分离的控制连接来传送命令，故选 A。

二、综合应用题

41.【解析】

该方法不一定能（或不能）求得最短路径。

例如，对于下图所示的带权图，如果按照题中的原则，从 A 到 C 的最短路径是 A->B->C，事实上其最短路径是 A->D->C。



42. 【解析】

(1) 算法的基本设计思想 (5 分) :

问题的关键是设计一个尽可能高效的算法, 通过链表的一趟遍历, 找到倒数第 k 个结点的位置。算法的基本设计思想是: 定义两个指针变量 p 和 q , 初始时均指向头结点的下一个结点 (链表的第一个结点)。 p 指针沿链表移动; 当 p 指针移动到第 k 个结点时, q 指针开始与 p 指针同步移动; 当 p 指针移动到最后一个结点时, q 指针所指示结点为倒数第 k 个结点。以上过程对链表仅进行一遍扫描。

(2) 算法的详细实现步骤 (5 分) :

① $\text{count}=0$, p 和 q 指向链表表头结点的下一个结点;

② 若 p 为空, 转⑤;

③ 若 count 等于 k , 则 q 指向下一个结点; 否则, $\text{count}=\text{count}+1$;

④ p 指向下一个结点, 转②;

⑤ 若 count 等于 k , 则查找成功, 输出该结点的 data 域的值, 返回 1; 否则, 说明 k 值超过了线性表的长度, 查找失败, 返回 0;

⑥ 算法结束。

(3) 算法实现 (5 分) :

```
typedef int ElemType;           // 链表数据的类型定义
```

```
typedef struct LNode{           // 链表结点的结构定义
```

```
    ElemType data;               // 结点数据
```

```
    struct LNode *link;          // 结点链接指针
```

```
} *LinkList;
```

```
int Search_k(LinkList list, int k){ // 查找链表 list 倒数第 k 个结点, 并输出该结点 data 域的
```

值

```
    LinkList p = list->link, q = list->link;           // 指针 p、q 指示第一个结点
```

```
    int count = 0;
```

```
    while(p != NULL){ // 遍历链表直到最后一个结点
```

```
        if(count < k) count++; // 计数, 若 count < k 只移动 p
```

```
        else q = q->link;
```

```
        p = p->link; // 之后让 p、q 同步移动
```

```
    } // while
```

```
    if(count < k) return 0; // 查找失败返回 0
```

```
    else { // 否则打印并返回 1
```

```
        printf("%d", q->data); return 1;
```

```
}
} // Search_k
```

43. 【解析】

(1) 按题意，外设每秒传送 0.5MB，中断时每次传送 4B。中断方式下，CPU 每次用于数据传输的时钟周期为： $5 \times 18 + 5 \times 2 = 100$ 。

为达到外设 0.5MB/s 的数据传输率，外设每秒申请的中断次数为： $0.5\text{MB}/4\text{B} = 125000$ 。

1 秒钟内用于中断的开销： $100 \times 125000 = 12500000 = 12.5\text{M}$ 个时钟周期。

CPU 用于外设 I/O 的时间占整个 CPU 时间的百分比： $12.5\text{M}/500\text{M} = 2.5\%$ 。

(2) 当外设数据传输率提高到 5MB/s 时改用 DMA 方式传送，每次 DMA 传送 5000B，1 秒钟内需产生的 DMA 次数： $5\text{MB}/5000\text{B} = 1000$ 。

CPU 用于 DMA 处理的总开销： $1000 \times 500 = 500000 = 0.5\text{M}$ 个时钟周期。

CPU 用于外设 I/O 的时间占整个 CPU 时间的百分比： $0.5\text{M}/500\text{M} = 0.1\%$ 。

44. 【解析】

一条指令的执行过程通常由取指、译码和执行 3 个步骤完成，本题中取指用 3 个节拍、译码用 1 个节拍，执行加法运算并把结果写入主存如何完成呢？包括划分执行步骤、确定完成的功能、要提供的控制信号，这是本题要测试的内容。为回答这个问题，首先要看清图中给出的部件组成情况和信息传送的路径。要完成的功能是 $(R0) + ((R1)) \rightarrow (R1)$ ，从图中看到：

(1) R0、R1 都有送自己内容到内总线的路径，控制信号分别是 R0out 和 R1out；

(2) ALU 加运算，2 个数据由工作寄存器 A 和内总线提供，控制信号是 Add；A 只接收内总线的内容，控制信号是 Ain；结果需存 AC，控制信号是 ACin；AC 的内容可送内总线，控制信号是 ACout；

(3) PC 可接收内总线的内容，还可增 1，控制信号是 PCin 和 PC+1，PC 的内容可送内总线，控制信号是 PCout；

(4) 指令寄存器 IR 可接收内总线的内容，控制信号是 IRin；

(5) 读写存储器时，地址由 MAR 经 AB 提供，MAR 只接收总线上的信息，控制信号是 MARin；

(6) 读存储器，提供读命令 MemR，并通过 DB 送入 MDR，控制信号是 MDRinE；MDR 的内容可送入总线，控制信号是 MDRout；

(7) 写存储器，提供写命令 MemW，数据由 MDR 通过 DB 送到存储器的数据引脚，控制信号是 MDRoutE；

然后是划分执行步骤、确定每一步完成的功能、需要提供的控制信号。这是由指令应完成的功能和计算机硬件的实际组成情况和信息传送的可用路径共同决定的，基本原则是步骤越少越好。硬件电路要能支持，可以有多种方案，解题时应参照以给出的答题格式，即取指和译码阶段的那张表的内容，但不必把表已有的内容再抄一遍。

划分指令执行步骤，确定每一步完成的功能、给出需要提供的控制信号：

请注意， $(R0) + ((R1))$ 表示：R0 寄存器的内容与 R1 作地址从主存中读出来的数据完成加法运算；而 $o(R1)$ 表示把 R1 的内容作为主存储器的地址完成写主存操作。为防止出现误解，题中

还特地对此作了文字说明。这条指令的功能是先到主存储器取一个数,之后运算,再将结果写回主存储器。

(1) 执行相加运算,需把存储器中的数据读出,为此首先送地址,将 R1 的内容送 MAR,控制信号是 R1out、MARin。

(2) 启动读主存操作,读出的内容送入 MDR,控制信号是 MemR、MDRinE。还可同时把 R0 的内容经内总线送入 A,用到的控制信号是 R0out、Ain。

(3) 执行加法运算,即 A 的内容与 MDR 的内容相加,结果保存到 AC,控制信号是 MDRout、Add、Acin。

(4) 要把 AC 的内容写入主存,由于 R1 的内容已经在 MAR 中,地址已经有了,但需要把写入的数据(已经在 AC 中)经内总线送入 MDR,控制信号是 ACout、MDRin。

(5) 给出写主存的命令,把 MDR 的内容经 DB 送存储器的数据线引脚,执行写操作,控制信号是 MDRoutE、MemW。这几个步骤是有先后次序的,前面的完成了,下一步才可以执行,也保证了不会产生硬件线路的冲突。请注意,使用最为频繁的是内总线,它在任何时刻只能接收一个输入数据,并且向内总线发送信息的电路只能以三态门器件连接到内总线,5 个向内总线发送信息的控制信号(ACout, PCout, R0out, R1out, MDRout)最多只能有一个为 1,其他 4 个必须全为 0,或者 5 个全为 0。

仔细看一下,发现可以把第 2 个步骤的操作划分到两个步骤中完成,一个步骤中安排 MDR 接收从存储器中读出的内容,到另外一个步骤实现 R0 的内容送入 A,这多用了个操作步骤,指令的执行速度会变慢。有些解题者在写存储器之前,还会再执行一次把 R1 的内容送 MAR,尽管无此必要,但不属于原理上的错误。

时钟	功能	有效控制信号
C5	$MAR \leftarrow (R1)$	R1out, MARin
C6	$MDR \leftarrow M(MDR)$ $A \leftarrow (R0)$	MemR, MDRinE, R0out, Ain
C7	$AC \leftarrow (MDR) + (A)$	MDRout, Add, A Cin
C8	$MDR \leftarrow (AC)$	ACout, MDRin
C9	$M(MAR) \leftarrow (MDR)$	MDRoutE, MemW

45. 【解析】

定义信号量 odd 控制 P1 与 P2 之间的同步; even 控制 P1 与 P3 之间的同步; empty 控制生产者与消费者之间的同步; mutex 控制进程间互斥使用缓冲区。程序如下:

```
semaphore odd = 0, even = 0, empty = N, mutex = 1;
```

```
P1(){
```

```
while(1){
```

```
    x = produce(); // 生成一个数
```

```
    P(empty); // 判断缓冲区是否有空单元
```

```
    P(mutex); // 缓冲区是否被占用 Put();
```

```
    V(mutex); // 释放缓冲区
```

```
    if(x%2 == 0)
```

```
        V(even); // 如果是偶数, 向 P3 发出信号
```



```

    Else V(odd); //如果是奇数, 向 P2 发出信号
}
}
P2() {
    while(1) {
        P(odd); //收到 P1 发来的信号, 已产生一个奇数
        P(mutex); //缓冲区是否被占用
        getodd();
        V(mutex); //释放缓冲区
        V(empty); //向 P1 发信号, 多出一个空单元
        countodd();
    }
}
P3() {
    while(1){
        P(even); //收到 P1 发来的信号, 已产生一个偶数
        P(mutex); //缓冲区是否被占用
        geteven();
        V(mutex); //释放缓冲区
        V(empty); //向 P1 发信号, 多出一个空单元
        counteven();
    }
}

```

46. 【解析】

(1) 根据页式管理的工作原理, 应先考虑页面大小, 以便将页号和页内位移分解出来。页面大小为 4KB, 即 2^{12} , 则得到页内位移占虚地址的低 12 位, 页号占剩余高位。可得三个虚地址的页号 P 如下 (十六进制的一位数字转换成 4 位二进制, 因此, 十六进制的低三位正好为页内位移, 最高位为页号):

2362H: P=2, 访问快表 10ns, 因初始为空, 访问页表 100ns 得到页框号, 合成物理地址后访问主存 100ns, 共计 $10ns+100ns+100ns=210ns$ 。

1565H: P=1, 访问快表 10ns, 落空, 访问页表 100ns 落空, 进行缺页中断处理 10^8ns , 访问快表 10ns, 合成物理地址后访问主存 100ns, 共计 $10ns+100ns+10^8ns+10ns+100ns=100000220ns$ 。

25A5H: P=2, 访问快表, 因第一次访问已将该页号放入快表, 因此花费 10ns 便可合成物理地址, 访问主存 100ns, 共计 $10ns+100ns=110ns$ 。

(2) 当访问虚地址 1565H 时, 产生缺页中断, 合法驻留集为 2, 必须从页表中淘汰一个页面, 根据题目的置换算法, 应淘汰 0 号页面, 因此 1565H 的对应页框号为 101H。由此可得 1565H 的物理地址为 101565H。

47. 【解析】

(1) CIDR 中的子网号可以全 0 或全 1, 但主机号不能全 0 或全 1。

因此若将 IP 地址空间 202.118.1.0/24 划分为 2 个子网, 且每个局域网需分配的 IP 地址个数不少于 120 个, 子网号至少要占用一位。

由 $2^6 - 2 < 120 < 2^7 - 2$ 可知, 主机号至少要占用 7 位。

由于源 IP 地址空间的网络前缀为 24 位, 因此主机号位数+子网号位数=8。

综上可得主机号位数为 7, 子网号位数为 1。

因此子网的划分结果为: 子网 1: 202.118.1.0/25, 子网 2: 202.118.1.128/25。

地址分配方案: 子网 1 分配给局域网 1, 子网 2 分配给局域网 2, 或子网 1 分配给局域网 2, 子网 2 分配给局域网 1。

(2) 由于局域网 1 和局域网 2 分别与路由器 R1 的 E1、E2 接口直接相连, 因此在 R1 的路由表中, 目的网络为局域网 1 的转发路径是直接通过接口 E1 转发, 目的网络为局域网 2 的转发路径是直接通过接口 E2 转发。由于局域网 1、2 的网络前缀均为 25 位, 因此它们的子网掩码均为 255.255.255.128。

根据题意, R1 专门为域名服务器设定了一个特定的路由表项, 因此该路由表项中的子网掩码应为 255.255.255.255。对应的下一跳转发地址是 202.118.2.2, 转发接口是 L0。根据题意, 到互联网的路由实质上相当于一个默认路由, 默认路由一般写作 0/0, 即目的地址为 0.0.0.0, 子网掩码为 0.0.0.0。对应的下一跳转发地址是 202.118.2.2, 转发接口是 L0。

综上可得到路由器 R1 的路由表为:

(若子网 1 分配给局域网 1, 子网 2 分配给局域网 2)

目的网络 IP 地址	子网掩码	下一跳 IP 地址	接口
202.118.1.0	255.255.255.128	——	E1
202.118.1.128	255.255.255.128	——	E2
202.118.3.2	255.255.255.255	202.118.2.2	L0
0.0.0.0	0.0.0.0	202.118.2.2	L0

(若子网 1 分配给局域网 2, 子网 2 分配给局域网 1)

目的网络 IP 地址	子网掩码	下一跳 IP 地址	接口
202.118.1.128	255.255.255.128	——	E1
202.118.1.0	255.255.255.128	——	E2
202.118.3.2	255.255.255.255	202.118.2.2	L0
0.0.0.0	0.0.0.0	202.118.2.2	L0

(3) 局域网 1 和局域网 2 的地址可以聚合为 202.118.1.0/24, 而对于路由器 R2 来说, 通往局域网 1 和 2 的转发路径都是从 L0 接口转发, 因此采用路由聚合技术后, 路由器 R2 到局域网 1 和局域网 2 的路由为:

目的网络 IP 地址	子网掩码	下一跳 IP 地址	接口
202.118.1.0	255.255.255.0	202.118.2.1	L0